NARWCDB SQL Schema Reference

This document describes the SQL Server database schema: tables, column types, foreign key constraints, and indexes. It covers the structure of the database. For the meaning of individual fields — valid values, survey context, validation rules, and known data quality issues — see `database_reference.md`.

Database Facts

Property	Value
Database name	`NARWCDB`
Server	SQL Server 2014 Express (or later)
Collation	`SQL_Latin1_General_CP1_CI_AS`
Schema scripts	`scripts/sql/schema/`
Lookup table CSVs	`data/tables/`

The collation is case-insensitive and accent-sensitive, matching the legacy SQL Server instance. It handles FILEID lookups correctly without explicit `UPPER()`/`LOWER()` calls and avoids collation conflicts when joining temporary tables that inherit the server default.

Tables

The database has two kinds of tables:

- Master — one row per survey event (sighting, effort record, leg marker, watch transition). This is the primary data table.
- Lookup tables (24 tables) — one row per valid code value. Each lookup table has a `Value` column (the code) and a `Description` column. Several tables carry additional domain-specific columns.

Master Table

Script: `schema/02_create_master_table.sql`

The Master table uses a surrogate `Master_ID` integer identity as its primary key. This provides a stable row handle for curation queries and deletes without requiring `(FILEID, EVENTNO)` to be unique — MATLAB validation catches duplicates before upload.

Column List

Column	SQL Type	NULL	Notes
Master_ID	`int IDENTITY(1,1)`	NOT NULL	Surrogate PK, clustered
FILEID	`varchar(20)`	NOT NULL	Survey file code; 6–8 chars in practice
EVENTNO	`int`	NOT NULL	Sequential event number within file
YEAR	`smallint`	NULL	4-digit year
MONTH	`tinyint`	NULL	1–16; FK → MONTH(Value)
DAY	`tinyint`	NULL	1–31
TIME	`int`	NULL	HHMMSS integer, UTC
S_TIME	`int`	NULL	Leg-start time, HHMMSS UTC
LAT_DD	`decimal(10,5)`	NULL	Platform latitude, decimal degrees N
LONG_DD	`decimal(10,5)`	NULL	Platform longitude, decimal degrees (negative = W)
S_LAT	`decimal(10,5)`	NULL	Exact sighting latitude (NLPSC/MassCEC)
S_LONG	`decimal(10,5)`	NULL	Exact sighting longitude
ALT	`decimal(8,2)`	NULL	Flight altitude in feet (aerial only)
HEADING	`smallint`	NULL	Platform heading, degrees 0–360
PLATFORM	`int`	NULL	FK → PLAT-FORM.Value
LEGNO	`smallint`	NULL	Leg number within file

Column	SQL Type	NULL	Notes
LEGTYPE	int	NULL	FK → LEGTYPE.Value
LEGSTAGE	int	NULL	FK → LEGSTAGE.Value
DDSOURCE	varchar(4)	NULL	FK → DDSOURCE.Value
IDSOURCE	varchar(4)	NULL	FK → IDSOURCE.Value
BLOCK	varchar(4)	NULL	FK → Block.Value
STRATUM	varchar(4)	NULL	FK → STRATUM.Value
STRIP	int	NULL	FK → STRIP.Value; distance-interval code
BEAUFORT	int	NULL	FK → Beaufort.Value; sea state 0–12
CLOUD	int	NULL	FK → Cloud.Value; cloud cover (oktas)
GLAREL	int	NULL	FK → GLARE.Value; glare severity, left side
GLARER	int	NULL	FK → GLARE.Value; glare severity, right side
WX	varchar(4)	NULL	FK → WX.Value; weather code
VISIBLTY	decimal(8,2)	NULL	Visibility in nautical miles
SURFTEMP	decimal(8,2)	NULL	Sea surface temperature, °C
SIGHTNO	int	NULL	Sighting number; NULL for non-sighting rows

Column	SQL Type	NULL	Notes
SPECCODE	varchar(8)	NULL	FK → SPECCODE.Value
TAXCODE	int	NULL	FK → TAXCODE.Value
NUMBER	int	NULL	Group size best estimate
NUMCALF	int	NULL	Calf count within group
CONFIDNC	int	NULL	FK → Confidnc.Value; size-estimate confidence
IDREL	int	NULL	FK → IDREL.Value; ID reliability
PHOTOS	int	NULL	FK → PHOTOS.Value; photo type code
ANGLEL	smallint	NULL	Declination angle to sighting, left side
ANGLER	smallint	NULL	Declination angle to sighting, right side
ANHEAD	int	NULL	FK → ANHEAD.Value; animal heading code
BEHAV1	int	NULL	FK → Behave.Value
BEHAV2	int	NULL	FK → Behave.Value
BEHAV3	int	NULL	FK → Behave.Value
BEHAV4	int	NULL	FK → Behave.Value
BEHAV5	int	NULL	FK → Behave.Value
BEHAV6	int	NULL	FK → Behave.Value

Column	SQL Type	NULL	Notes
BEHAV7	int	NULL	FK → Behave.Value
BEHAV8	int	NULL	FK → Behave.Value
BEHAV9	int	NULL	FK → Behave.Value
BEHAV10	int	NULL	FK → Behave.Value
BEHAV11	int	NULL	FK → Behave.Value
BEHAV12	int	NULL	FK → Behave.Value
BEHAV13	int	NULL	FK → Behave.Value
BEHAV14	int	NULL	FK → Behave.Value
BEHAV15	int	NULL	FK → Behave.Value

Total: 56 columns (1 surrogate PK + 55 data fields).

Only FILEID and EVENTNO are `NOT NULL`. All other fields accept NULL because not every field applies to every event type — a non-sighting effort record has no SIGHTNO or SPECCODE; a ship survey has no ALT. Conditionally required fields (SIGHTNO, SPECCODE, LAT_DD, LONG_DD, TAXCODE, TIME, STRIP) cannot be declared NOT NULL at the column level because their requirement depends on record type — that's enforced by MATLAB validation instead (`required_fields.m` and friends), not the schema. See `PROJECT_STATUS.md` §7 for an open question about exactly which fields that rule currently treats as required.

Lookup Tables

Script: `schema/03_create_lookup_tables.sql`
Data: `data/tables/*.csv`, loaded by `schema/06_populate_lookup_tables.sql`

All lookup tables have a `PRIMARY KEY` on `Value`. The `Value` column type matches the corresponding Master column type exactly (required for FK constraints to work without implicit conversion).

Table	Value type	Master column(s)	Extra columns	Rows
ANHEAD	`int`	ANHEAD	Direction, LowDeg, HighDeg	20
Beaufort	`int`	BEAUFORT	lWind, hWind, Waves, Description	13
Behave	`int`	BEHAV1–BEHAV15	Description	90
Block	`varchar(4)`	BLOCK	Description	55
Cloud	`int`	CLOUD	Description	6
Confidnc	`int`	CONFIDNC	Description	12
Contrib	`varchar(2)`	(none — not FK'd)	Description	23
DDSOURCE	`varchar(4)`	DDSOURCE	Description	48
DType	`varchar(2)`	(none — not FK'd)	Description	5
GLARE	`int`	GLAREL, GLARER	Description	4
IDREL	`int`	IDREL	Description	4
IDSOURCE	`varchar(4)`	IDSOURCE	Description	53
LEGGOOD	`varchar(2)`	(none — not FK'd)	Description	2
LEGSTAGE	`int`	LEGSTAGE	Description	9
LEGTYPE	`int`	LEGTYPE	Description	9
MONTH	`tinyint`	MONTH	Description	16
OLDVIZ	`varchar(2)`	(none — retired field)	Description	5
PHOTOS	`int`	PHOTOS	Description	5
PLATFORM	`int`	PLATFORM	Description	283
SPECCODE	`varchar(8)`	SPECCODE	SPECNAME, SPECCHAR, SPECNUM, Type, TAXCODE, typical_max_group, typical_max_calf	317
STRATUM	`varchar(4)`	STRATUM	Description	10
STRIP	`int`	STRIP	Description	16
TAXCODE	`int`	TAXCODE	Description, typical_max_group, typical_max_calf	10
WX	`varchar(4)`	WX	Description	12

Notes on specific tables

ANHEAD has three extra columns (`Direction`, `LowDeg`, `HighDeg`) describing the compass segment each code represents. Useful for producing human-readable output without a separate reference document.

Beaufort has wind speed bounds (`lWind`, `hWind` in knots) and wave height

(Waves in meters) in addition to Description. Descriptions are long (up to ~400 characters); the Description column is varchar(500) in this table only.

SPECCODE carries the full species metadata: SPECNAME (full common name), SPECCHAR (2-char abbreviation), SPECNUM (numeric code), Type (broad category such as BIR, CETACEAN), and TAXCODE (the taxonomic group code linking back to the TAXCODE table). The TAXCODE column in SPECCODE is informational — it is not a foreign key constraint in the schema. typical_max_group and typical_max_calf are per-species warning thresholds used by species_rules.m; NULL means no species-specific override (the TAXCODE-level threshold or global default applies).

TAXCODE carries typical_max_group and typical_max_calf thresholds that apply to all species in that taxonomic group when no SPECCODE-level override is set. Curators adjust thresholds by editing data/tables/SPECCODE.csv or TAXCODE.csv and running push_lookup_tables.m — no code change needed.

Contrib, DType, LEGGOOD, OLDVIZ exist in the legacy database and are retained here but are not FK'd from Master. OLDVIZ covers the retired negative-VISIBLTY codes (−1 to −5) that appear in pre-2020 archived data.

MONTH has 16 rows: values 1–12 for calendar months and values 13–16 for season codes (Winter, Spring, Summer, Fall). FK constraint FK_Master_MONTH references MONTH(Value). MATLAB validation accepts 1–16 to match.

Foreign Key Constraints

Script: schema/05_add_foreign_keys.sql

35 constraints from Master columns to lookup table Value columns. All use WITH CHECK (the default), which verifies that existing rows satisfy the constraint at the time it is added. Because MATLAB validation rejects invalid codes before upload, this should always succeed on a clean database. If it fails, investigate the offending rows before switching to WITH NOCHECK.

Constraint	Master column	References
FK_Master_ANHEAD	ANHEAD	ANHEAD(Value)
FK_Master_BEAUFORT	BEAUFORT	Beaufort(Value)
FK_Master_BEHAV1	BEHAV1	Behave(Value)
FK_Master_BEHAV2	BEHAV2	Behave(Value)
FK_Master_BEHAV3	BEHAV3	Behave(Value)
FK_Master_BEHAV4	BEHAV4	Behave(Value)
FK_Master_BEHAV5	BEHAV5	Behave(Value)
FK_Master_BEHAV6	BEHAV6	Behave(Value)

Constraint	Master column	References
FK_Master_BEHAV7	BEHAV7	Behave(Value)
FK_Master_BEHAV8	BEHAV8	Behave(Value)
FK_Master_BEHAV9	BEHAV9	Behave(Value)
FK_Master_BEHAV10	BEHAV10	Behave(Value)
FK_Master_BEHAV11	BEHAV11	Behave(Value)
FK_Master_BEHAV12	BEHAV12	Behave(Value)
FK_Master_BEHAV13	BEHAV13	Behave(Value)
FK_Master_BEHAV14	BEHAV14	Behave(Value)
FK_Master_BEHAV15	BEHAV15	Behave(Value)
FK_Master_BLOCK	BLOCK	Block(Value)
FK_Master_CLOUD	CLOUD	Cloud(Value)
FK_Master_CONFIDNC	CONFIDNC	Confidnc(Value)
FK_Master_DDSOURCE	DDSOURCE	DDSOURCE(Value)
FK_Master_GLAREL	GLAREL	GLARE(Value)
FK_Master_GLARER	GLARER	GLARE(Value)
FK_Master_IDREL	IDREL	IDREL(Value)
FK_Master_IDSOURCE	IDSOURCE	IDSOURCE(Value)
FK_Master_LEGSTAGE	LEGSTAGE	LEGSTAGE(Value)
FK_Master_LEGTYPE	LEGTYPE	LEGTYPE(Value)
FK_Master_MONTH	MONTH	MONTH(Value)
FK_Master_PHOTOS	PHOTOS	PHOTOS(Value)
FK_Master_PLATFORM	PLATFORM	PLATFORM(Value)
FK_Master_SPECCODE	SPECCODE	SPECCODE(Value)
FK_Master_STRATUM	STRATUM	STRATUM(Value)
FK_Master_STRIP	STRIP	STRIP(Value)
FK_Master_TAXCODE	TAXCODE	TAXCODE(Value)
FK_Master_WX	WX	WX(Value)

To drop all FK constraints at once (reversal):

```
DECLARE @sql nvarchar(max) = '';
SELECT @sql = @sql + 'ALTER TABLE Master DROP CONSTRAINT '
    + QUOTENAME(name) + ';' + CHAR(13)
FROM sys.foreign_keys WHERE parent_object_id = OBJECT_ID('Master');
EXEC sp_executesql @sql;
```

Indexes

Script: `schema/04_create_indexes.sql`

The `Master_ID` clustered primary key is already an index. Five additional non-clustered indexes cover the most common query patterns:

Index	Column(s)	Rationale
IX_Master_FILEID	FILEID	Survey-level deletes and lookups (`WHERE FILEID = '...'`)
IX_Master_YEAR	YEAR	Date-range queries
IX_Master_SPECCODE	SPECCODE	Species-level queries and reporting
IX_Master_LAT_LONG	LAT_DD, LONG_DD	Bounding-box spatial filtering
IX_Master_PLATFORM	PLATFORM	Platform-filtered queries

These are intentionally conservative. Each non-clustered index adds overhead to every `INSERT` during batch migration. Add indexes for specific query patterns once the query shapes are known from production use.

Type Design Decisions

Why **`varchar`** for some coded fields?

Fields typed `varchar` in Master are genuinely alphanumeric — they contain letter codes in the real data:

- SPECCODE: codes like `RIWH`, `AC-J`, `FG-A` — letter codes are the norm.
- WX: single-letter codes `B`, `C`, `D`, `F`, etc.
- DDSOURCE / IDSOURCE: 3–4 letter abbreviations like `ASW`, `CCS`, `CAM`.
- STRATUM: mix of digits and letters: `0`, `A`, `B`, `I`, `X`, `Y`, `Z`.
- BLOCK: typed as `string` in `FieldDefinitions.m`; some historical block codes may be non-numeric.
- FILEID: an 8-character alphanumeric survey identifier.

Why **`int`** for other coded fields?

Fields typed `int` hold integer codes whose values are always numeric in the CSV data and in `FieldDefinitions.m` (where they appear as `double`):

PLATFORM, CLOUD, IDREL, CONFIDNC, ANHEAD, BEAUFORT, GLAREL, GLARER, LEGTYPE, LEGSTAGE, PHOTOS, STRIP, TAXCODE, BEHAV1–15.

Using `int` keeps the FK-join types consistent, avoids implicit string conversion on every insert, and matches what MATLAB stores internally.

Why **`decimal(10,5)`** for coordinates?

Decimal degrees to 5 places gives ~1 meter precision at any latitude. `float` would work but introduces binary rounding artifacts in display and comparison. `decimal` is exact and predictable for coordinate data.

Why a surrogate PK instead of **`(FILEID, EVENTNO)`**?

`(FILEID, EVENTNO)` is the logical key but is not enforced as unique at the database level because a small number of historical surveys have genuine duplicate `(FILEID, EVENTNO)` pairs (known data quality issue — see README). A surrogate `Master_ID` gives every row a stable identifier for curation operations (targeted deletes, audit references) without requiring the data to be clean enough for a composite unique constraint.

Relationship to MATLAB Code

`src/+narwc/+db/FieldDefinitions.m` is the canonical source of field names and their MATLAB types. The SQL schema was built to match it. Where the two can diverge:

- MATLAB `double` → SQL `int`, `smallint`, `tinyint`, or `decimal` depending on the field's value range and precision needs.
- MATLAB `string` → SQL `varchar(N)` where N is chosen from observed data widths.
- The SQL schema does not know about MATLAB validation logic; it relies on MATLAB having already rejected invalid codes before upload.

The MATLAB `Connection` class (`src/+narwc/+db/Connection.m`) handles the actual INSERT statements and type mapping. If a column type changes in the SQL schema, the corresponding MATLAB insert logic may need updating.

Known FieldDefinitions.m inaccuracies

The SQL schema was derived from `FieldDefinitions.m` but the reference doc (`database_reference.md`) documents several cases where `FieldDefinitions.m` carries incorrect metadata. The SQL schema reflects the correct values:

Field	FieldDefinitions.m says	Actual (per reference doc)
BEAUFORT	range 0–9	range 0–12 (full Beaufort scale)
CLOUD	range 0–10	range 0–8 (oktas scale)
ALT	units unspecified / implied meters	feet (verified against survey data)

These are metadata errors in the MATLAB file, not type errors — the SQL `int` and `decimal` column types are correct in all three cases. The discrepancies matter if you are using `FieldDefinitions.m` to drive range validation; consult `database_reference.md` for authoritative value ranges.